TAAC Checkpoint 2: divisão de tarefas

Nov 8, 2026 · @Afonso

Objetivo e narrativa

No Checkpoint 2 (11 de novembro) apresentamos a limitação com evidência empírica e depois a proposta com a matemática. O Fishr-EI não precisa de estar implementado; os resultados que mostramos são dos baselines e da limitação.

A apresentação segue esta ordem, e cada pessoa produz a evidência de um ou mais passos:

1. **Validação da base:** os nossos números do Fishr batem com os do artigo, logo a implementação de partida está correta. (Pessoa B, Pessoa C)
2. **Limitação 1, do Fishr:** sem etiquetas de ambiente, o Fishr com divisão aleatória colapsa para ERM, no Colored MNIST e no Waterbirds. (Pessoa B, Pessoa C)
3. **Limitação 2, da solução existente (EIIL):** inferir ambientes a partir dos erros do ERM funciona com ruído alto nas etiquetas e falha com ruído médio. (Pessoa B)
4. **Ponte para a proposta:** com ruído médio, a diferença das médias dos gradientes entre grupos desaparece, mas a diferença das variâncias mantém-se. (Pessoa B mede, Pessoa A interpreta)
5. **Proposta Fishr-EI:** formulação, motivação teórica, originalidade e protocolo experimental da fase 3. (Pessoa A)

Cuidado no ponto 3: a limitação é do EIIL, não do Fishr. Apresentamo-la como "a solução atual para a limitação do Fishr também falha".

Organização do repositório

Um repositório Git do grupo, com uma pasta por bloco de trabalho. Cada pessoa só escreve na sua pasta e em `results/`, por isso ninguém tem conflitos de merge. Os repositórios externos (Fishr, EIIL) são clonados para `external/` e não se editam; o nosso código chama-os ou copia o que precisa.

```
taac-fishr-ei/
├── README.md                  # como instalar e correr tudo, por ordem
├── environment.yml            # ambiente conda principal (Python 3.10+)
├── environment-fishr.yml      # ambiente do repo oficial do Fishr (Python
3.7, torch 1.8.1)
├── .gitignore                 # ignora data/, external/, *.pt, *.npy
│
├── external/                  # clones dos repos oficiais, NÃO editar (fora
```

```
do git)
|   ├── fishr/                # alexrame/fishr
|   └── eiil/                 # ecreager/eiil
|
├── shared/                   # código comum, combinado na segunda de manhã
|   ├── results_io.py         # escrever/ler o CSV de resultados (formato
único)
|   ├── seeds.py              # lista de seeds e set_seed()
|   └── plotting.py           # estilo comum dos gráficos
|
├── theory/                   # Pessoa A
|   ├── fishr_ei_formulation.md  # formulação, derivações, motivação teórica
|   ├── literature.md           # tabela de literatura e argumento de
originalidade
|   ├── experimental_protocol.md # protocolo planeado para a fase 3
|   └── toy_2d.ipynb            # exemplo sintético: médias iguais,
variâncias diferentes
|
├── cmnist/                   # Pessoa B
|   ├── reproduce_fishr.sh      # corre ERM/IRM/V-REx/Fishr do repo oficial
|   ├── random_envs.py          # Fishr com divisão aleatória dos ambientes
|   ├── eiil_noise_sweep.py     # EIIL→Fishr ao longo do ruído nas etiquetas
|   ├── grad_diagnostic.py      # diferença de médias vs variâncias dos
gradientes
|   └── analysis.ipynb          # gráficos e tabelas do Colored MNIST
|
├── waterbirds/               # Pessoa C
|   ├── download_data.py        # Waterbirds via pacote wilds
|   ├── extract_features.py     # features ResNet-50 →
data/waterbirds_feats/*.npy
|   ├── train_last_layer.py     # ERM / Fishr / Fishr aleatório sobre as
features
|   └── analysis.ipynb          # tabela de média e worst-group accuracy
|
├── data/                     # dados e features (fora do git)
├── results/
|   ├── raw/                  # um CSV por experiência, formato único
|   └── figures/              # PNG/PDF finais para os slides
└── notebooks/
    └── checkpoint2_summary.ipynb  # junta todos os CSV e gera as figuras
finais
```

Regras comuns (combinar segunda de manhã, 30 min)

Esta reunião é o único ponto em que alguém espera por alguém. Saem dela três coisas fechadas:

- **Formato de resultados.** Todas as experiências escrevem linhas com estas colunas, através de `shared/results_io.py`:

```
dataset,method,setting,seed,train_acc,test_acc,worst_group_acc,partition_puri
ty
cmnist,fishr,noise=0.25,0,0.710,0.695,,
cmnist,fishr_random_envs,noise=0.25,0,0.851,0.152,,
waterbirds,erm,default,0,0.982,0.861,0.612,
```

- **Seeds.** Colored MNIST: 10 seeds (é o que o repo oficial usa e permite comparar média ± desvio com o artigo). Waterbirds: 5 seeds. A lista fica em `shared/seeds.py`.
- **Seleção de modelo.** Hiperparâmetros escolhidos com validação dos dados de treino, nunca com o teste. Onde usarmos os hiperparâmetros do artigo, dizemos isso explicitamente.

Os valores no CSV acima são só exemplo de formato, não resultados.

Pessoa A: matemática, proposta e literatura

Não depende de ninguém: trabalha com os artigos e com dados sintéticos. Só a secção da motivação empírica usa o diagnóstico da Pessoa B, e pode ser escrita com um espaço reservado até terça à noite.

1. Formalizar o Fishr (segunda). Para cada exemplo i do ambiente e, o gradiente da perda na última camada é g_i. A penalização compara a variância desses gradientes entre ambientes:

```
v_e = \frac{1}{n_e}\sum_{i \in e} \left(g_i - \bar g_e\right)^2, \qquad
\mathcal{L}_{\text{Fishr}} = \sum_{e} \mathcal{R}_e + \lambda \sum_{e}
\left\lVert v_e - \bar v \right\rVert^2
```

2. Formalizar o EIIL (segunda). Com o modelo de referência fixo, otimiza atribuições suaves $q_i \in [0, 1]$ para maximizar a penalização do IRMv1. Mostrar onde entra a dependência do modelo de referência.

3. Formular o Fishr-EI (segunda/terça). Mesmo esquema, mas o critério é a diferença de variâncias, com um termo que impede grupos degenerados (ex.: um grupo com meia dúzia de exemplos extremos):

```
\max_{q \in [0,1]^N} \; \left\lVert v_A(q) - v_B(q) \right\rVert^2 - \beta
\left(\tfrac{1}{N}\textstyle\sum_i q_i - \tfrac{1}{2}\right)^2, \quad v_A(q)
= \frac{\sum_i q_i (g_i - \bar g_A)^2}{\sum_i q_i}
```

Com v_B igual, usando 1 − q_i. Derivar também a forma fechada do gradiente na última camada de um classificador binário:

```
g_i = (p_i - y_i)\, z_i
```

onde z_i é a representação e p_i a probabilidade prevista. Isto mostra que os gradientes calculam-se uma vez só e a inferência é barata.

4. Motivação teórica (terça). O IRMv1 usa o primeiro momento dos gradientes (média), o Fishr-EI usa o segundo (variância). Construir no `toy_2d.ipynb` um exemplo com uma variável causal e uma espúria em que as médias dos dois grupos coincidem e as variâncias não. Figura para os slides.

5. Revisão de literatura e originalidade (terça). Tabela em `literature.md` com colunas: precisa de etiquetas de ambiente? critério usado? benchmarks? Artigos mínimos: IRM, V-REx, Fishr, EIIL, Repeated Environment Inference, Decorr, Invariant Learning with Annotation-free Environments (2025), JTT, DFR, In Search of Lost Domain Generalization. Confirmar que ninguém usou a penalização do Fishr para inferir ambientes.

6. Protocolo planeado para a fase 3 (terça). Benchmarks, métodos a comparar (ERM, Fishr com ambientes verdadeiros, Fishr aleatório, EIIL→Fishr, Fishr-EI), métricas (exatidão de teste, worst-group accuracy, pureza da partição), seeds, seleção de modelo.

Entrega (quarta de manhã): `fishr_ei_formulation.md`, `literature.md`, `experimental_protocol.md`, figura do exemplo 2D.

Pessoa B: Colored MNIST

Parte do código oficial do Fishr e do EIIL, não implementa nada de raiz. O que for preciso alterar copia-se para `cmnist/` e edita-se lá, nunca em `external/`.

1. Ambiente e reprodução (segunda de manhã, primeira coisa). O repo do Fishr pede versões antigas (Python 3.7.10, torch 1.8.1, backpack-for-pytorch 1.3.0), por isso cria-se um ambiente conda à parte (`environment-fishr.yml`). Depois:

```
for algo in erm irm rex fishr; do
  python3 coloredmnist/train_coloredmnist.py --algorithm $algo
done
```

Confirmar no código os nomes exatos dos algoritmos. O script já faz a média de 10 corridas. Comparar com a tabela da secção "Prova de reprodução".

2. Limitação 1: Fishr com ambientes aleatórios (segunda). Copiar `train_coloredmnist.py` para `cmnist/random_envs.py` e baralhar os exemplos entre os dois ambientes de treino antes de treinar. Resultado esperado: perto do ERM. Correr com o ruído por defeito (0.25).

3. Limitação 2: EIIL ao longo do ruído (segunda à tarde e terça). O repo do EIIL já tem um varrimento do ruído nas etiquetas no Colored MNIST (`exps/cmnist_label_noise_sweep.sh`, gráfico em `notebooks/plot_cmnist_label_noise_sweep.ipynb`), mas usa IRM no segundo passo.

- Primeiro: correr o varrimento tal como está (EIIL→IRM). Já mostra a queda na zona de ruído médio e é a versão de segurança se o resto não der tempo.
- Depois: guardar as atribuições de ambiente inferidas pelo EIIL e usá-las para treinar o Fishr (EIIL→Fishr).
- Para comparar: Fishr com ambientes verdadeiros ao longo do mesmo ruído, usando `--label_flipping_prob` do script do Fishr.
- Valores de ruído: 0.05, 0.10, 0.15, 0.20, 0.25.

4. Diagnóstico de gradientes, a ponte para a proposta (terça). Para cada nível de ruído:

1. Treinar o ERM de referência (o mesmo do EIIL).
2. Dividir os exemplos pelos grupos verdadeiros: cor concorda / cor discorda da etiqueta.
3. Calcular os gradientes da última camada por exemplo e medir, entre os dois grupos, a norma da diferença das médias e a norma da diferença das variâncias.
4. Normalizar as duas curvas pelo valor a ruído 0.25 para ficarem na mesma escala.

Se a diferença das médias cair com ruído médio e a das variâncias se mantiver, é a motivação empírica do Fishr-EI. Se ambas caírem, avisar logo a Pessoa A: a proposta tem de ser ajustada antes de quarta.

Entrega (quarta de manhã): tabela de reprodução, gráfico exatidão de teste vs ruído (ERM, Fishr com ambientes verdadeiros, Fishr aleatório, EIIL→IRM, EIIL→Fishr), gráfico do diagnóstico de gradientes, pureza das partições do EIIL por nível de ruído.

Pessoa C: Waterbirds

Mostra que a limitação 1 existe em imagens reais, não só no sintético. Pássaros terrestres vs aquáticos, com o fundo (terra/água) como atalho; cerca de 4800 imagens de treino. Para caber em dois dias, treina-se só a última camada sobre features fixas de uma ResNet-50 pré-treinada. É coerente com o Fishr, cuja penalização já atua só na última camada.

Atenção: o artigo do Fishr não tem resultados no Waterbirds (só Colored MNIST e DomainBed). A validação aqui não é "bate com o Fishr", é "o ERM se comporta como na literatura". Ver a secção seguinte.

Escolha do dataset: o Waterbirds com features fixas é muito mais leve que o VLCS do DomainBed (uma passagem pela ResNet-50 e depois uma regressão logística que treina em segundos, contra fine-tuning completo da ResNet-50 por seed). Fica o Waterbirds para o Checkpoint 2; o VLCS, onde há números do Fishr para reproduzir, entra na fase 3 se houver tempo.

1. Dados e features (segunda, primeira coisa). Descarregar o Waterbirds pelo pacote `wilds` (ou o tarball do repo group_DRO). Extrair as features da ResNet-50 pré-treinada no ImageNet (camada antes da classificação, 2048 dimensões) para treino, validação e teste, e guardar em `data/waterbirds_feats/`. É o único passo pesado: fazer na GPU logo de manhã.

2. Ambientes verdadeiros (segunda). O Waterbirds tem grupos (tipo de pássaro × fundo) nos metadados, mas não tem dois ambientes como o Colored MNIST. Construir dois ambientes de treino com força de atalho diferente, distribuindo os exemplos dos grupos minoritários de forma desigual (ex.: a maioria dos minoritários no ambiente 2). Documentar a regra exata, porque a professora vai perguntar.

3. Baselines (terça), 5 seeds, sobre as features:

- ERM: valida o pipeline (exatidão média alta, worst-group baixa).
- Fishr com os ambientes do passo 2: deve melhorar a worst-group face ao ERM.
- Fishr com ambientes aleatórios: limitação 1, deve ficar perto do ERM.
- EIIL→Fishr: só se sobrar tempo. O EIIL já foi avaliado no Waterbirds no artigo original, por isso não é crítico.

O Fishr sobre features é uma regressão logística com a penalização; dá para implementar em PyTorch em poucas linhas a partir da `coloredmnist/train_coloredmnist.py` do repo oficial, sem precisar do BackPACK (o gradiente por exemplo na última camada tem forma fechada).

4. Seleção de modelo. λ do Fishr e regularização escolhidos com a worst-group accuracy da validação, nunca do teste.

Entrega (quarta de manhã): tabela com exatidão média e worst-group accuracy (média ± desvio) para ERM, Fishr com ambientes, Fishr aleatório, e a regra de construção dos ambientes.

Prova de reprodução

Objetivo: um slide com "artigo vs nós" que mostra que a base está bem implementada antes de criticarmos o método. Critério para dizer que reproduz: a nossa média fica dentro de cerca de 2 desvios padrão da do artigo. Se não ficar, reportamos a diferença e uma explicação plausível, sem esconder.

Colored MNIST, setup do IRM, ruído 0.25, média de 10 corridas (README do repo oficial, Tabela 2 do artigo). Responsável: Pessoa B.

Método	Treino, artigo (%)	Teste, artigo (%)	Treino, nós (%)	Teste, nós (%)
ERM	86.4 ± 0.2	14.0 ± 0.7		
IRM	71.0 ± 0.5	65.6 ± 1.8		
V-REx	71.7 ± 1.5	67.2 ± 1.5		
Fishr	71.0 ± 0.9	69.5 ± 1.0		

Não confundir com os números do Colored MNIST no DomainBed (outro protocolo, outros valores). Usamos só os desta tabela.

EIIL no Colored MNIST, ruído 0.25 (artigo do EIIL, Tabela 2). Responsável: Pessoa B. O EIIL→IRM reportado é de cerca de 68.4% no teste, contra cerca de 65.5% do IRM com ambientes verdadeiros. Confirmar estes dois valores diretamente no PDF antes de os pôr num slide.

Waterbirds. Não há números do Fishr para comparar. Responsável: Pessoa C. Validação do pipeline: o ERM tem de mostrar o padrão conhecido (exatidão média alta, worst-group muito mais baixa). Se a Pessoa C quiser um número externo, usar o ERM com features fixas reportado no artigo do DFR, citando a tabela exata.

Calendário e riscos

Tudo o que está na mesma linha corre em paralelo. O único momento em que alguém pode esperar é terça à noite, quando a Pessoa A recebe o diagnóstico de gradientes da Pessoa B, e tem trabalho para fazer até lá.

Quando	Pessoa A	Pessoa B	Pessoa C
Seg, manhã	Reunião de 30 min: regras comuns e repo	idem	idem

Quando	Pessoa A	Pessoa B	Pessoa C
Seg	Formalizar Fishr, EIIL e Fishr-EI	Ambiente conda + reprodução do Fishr; Fishr com ambientes aleatórios	Download + extração de features; construir ambientes
Ter	Exemplo 2D, literatura, protocolo da fase 3	Varrimento EIIL ao longo do ruído; diagnóstico de gradientes	ERM, Fishr e Fishr aleatório, 5 seeds
Ter, noite	Integra o diagnóstico na motivação	Envia gráficos para `results/figures/`	Envia tabela para `results/figures/`
Qua, manhã	Cada um explica o seu bloco aos outros em 5 min	idem	idem

Riscos e plano B:

- [ ] **Ambiente do Fishr não instala** (Python 3.7, torch 1.8.1). Limite de meio dia; se falhar, a Pessoa B reimplementa a penalização em torch atual sobre o código do Colored MNIST do IRM, que é um MLP pequeno.
- [ ] **Reprodução não bate.** Não perder mais de meio dia; reportar a diferença e seguir.
- [ ] **EIIL→Fishr não fica pronto.** Usar o varrimento EIIL→IRM tal como sai do repo; a queda na zona de ruído médio já demonstra a limitação 2.
- [ ] **Download do Waterbirds falha.** Usar o mirror no Hugging Face ou o tarball do group_DRO.
- [ ] **Diagnóstico de gradientes não mostra o padrão esperado.** Reunião rápida terça à noite para ajustar a proposta antes de quarta.

Links

Repositórios públicos

Repositório	Para quê	Quem usa
alexrame/fishr	Código oficial do Fishr: Colored MNIST (`coloredmnist/train_coloredmnist.py`) e DomainBed	B, C
ecreager/eiil	Código oficial do EIIL, inclui varrimento do ruído no Colored MNIST	B

Repositório	Para quê	Quem usa
facebookresearch/InvariantRiskMinimization	Código original do IRM e do Colored MNIST	B
kohpangwei/group_DRO	Dataset Waterbirds (tarball) e definição dos grupos	C
p-lambda/wilds	Pacote que descarrega o Waterbirds automaticamente	C
PolinaKirichenko/deep_feature_reweighting	DFR: retreino da última camada no Waterbirds, referência para o pipeline sobre features	C, A

Artigos

- Fishr: Invariant Gradient Variances for Out-of-Distribution Generalization (ICML 2022)
- Invariant Risk Minimization (2019)
- Environment Inference for Invariant Learning, EIIL (ICML 2021)
- Last Layer Re-Training is Sufficient for Robustness to Spurious Correlations, DFR (ICLR 2023)
- Invariant Learning with Annotation-free Environments (2025)
- Distributionally Robust Neural Networks for Group Shifts, origem do Waterbirds